Trabajo Práctico 2

1. Probar que el lenguaje $L_U = \{(\langle M \rangle, w) \mid M \text{ acepta } w\}$ pertenece a la clase RE. Comentario: probarlo construyendo una MT.

Para probar que el lenguaje L_U descripto pertenece a la clase RE, se debe construir una MT que lo acepte pero no necesariamente se detenga siempre para las cadenas que no son parte del lenguaje. Sea la MT construida T:

1. T recibe una cadena de la forma $(\langle M \rangle, w)$, donde M es una MT y w es una cadena.
2. Si M no es una MT válida, T rechaza.
3. Si M es una MT válida, T simula la ejecución de M sobre la cadena w.
4. Si durante la simulación M **acepta** w, entonces T acepta la cadena de entrada $(\langle M \rangle, w)$.
5. Si durante la simulación M **rechaza** o **loopea** w, entonces T no acepta la cadena de entrada $(\langle M \rangle, w)$, ya sea rechazando o loopeando.

De esta forma se cumple que si $(\langle M \rangle, w) \in L_U$, entonces M acepta, y si $(\langle M \rangle, w) \notin L_U$, entonces M no acepta, ya sea rechazando o loopeando. Por lo tanto, $L_U \in RE$.

2. Responder breve y claramente cada uno de los siguientes incisos (en todos los casos, las MT mencionadas tienen una sola cinta):

a. Probar que se puede decidir si una MT M, a partir de la cadena vacía λ, escribe alguna vez un símbolo no blanco. Ayuda: ¿Cuántos pasos puede hacer M antes de entrar en un loop?

Se tiene una MT M que empieza con infinitos símbolos blancos a izquierda y derecha en la cinta. Se quiere chequear si M escribe algún símbolo que no sea el blanco en la cinta o no.

Ejemplo 1 (la MT entra en loop):

1. $\delta(q_0, B) = (q_0, B, R)$

Como todo en la cinta es blanco, la MT loopea moviéndose a la derecha infinitamente sin escribir ningún símbolo que no sea el blanco.

Ejemplo 2 (la MT escribe un símbolo que no es blanco):

1. $\delta(q_0, B) = (q_1, a, R)$

En este caso, la MT escribe el símbolo a que no es blanco, y luego se mueve a la derecha. En este caso, la MT no loopea, ya que no vuelve a q_0.

Se puede construir una MT T que decide si la MT M escribe alguna vez un símbolo no blanco a partir de la cadena vacía λ de la siguiente forma:

1. Simula la ejecución de M a partir de la cadena vacía λ. El estado inicial de M es q_0 y el cabezal está en la posición inicial, como de costumbre.
2. En cada paso, T lee el símbolo actual al que está apuntando el cabezal de M. Mientras no se haya escrito ningún símbolo distinto de blanco, todos los símbolos que se lean serán blancos ya que la cinta permanece totalmente blanca.
3. Por lo anterior, cada paso depende únicamente del estado actual + el símbolo blanco. Si la transición $\delta(q, B)$ no está definida, M se detiene sin haber escrito un símbolo no blanco, y por lo tanto T rechaza.
4. Si la transición $\delta(q, B)$ está definida, es decir tenemos $\delta(q, B) = (q', s, d)$:
   i. Si $s \neq B$, entonces M escribe un símbolo no blanco, y por lo tanto T acepta.
   ii. Si $s = B$, entonces se pasa al estado q' y se continúa simulando.
5. Como el conjunto de estados Q de M es **finito**, la secuencia de estados por la que pasa M (mientras solo lee blancos) debe repetirse luego de, como mucho, $|Q| + 1$ pasos. Si se repite un estado sin jamás haber escrito un símbolo no blanco, entonces la MT M entró en un loop y por ende nunca escribirá un símbolo no blanco, y entonces T rechaza.

Como se ha construido una MT T que **acepta** si M escribe alguna vez un símbolo no blanco a partir de la cadena vacía λ, y **rechaza** en caso contrario y además nunca loopea, se concluye que el problema es decidible.

b. Probar que se puede decidir si una MT M que sólo se mueve a la derecha, a partir de una cadena w, para, Ayuda:

¿Cuántos pasos puede hacer M antes de entrar en un loop?

Se tiene una MT M que sólo puede moverse a la derecha, y se quiere decidir si M para a partir de una cadena w.

Se sabe que:

1. Las cadenas de entrada w son **finitas** y tienen una longitud $n = |w|$.
2. Luego del final de la cadena w, la cinta contiene infinitos símbolos blancos hacia la derecha.
3. La MT solo se puede mover a la derecha, por lo tanto, una vez que el cabezal de la MT se mueve más allá del final de la cadena w, solo leerá símbolos blancos. Además, debido a esta restricción se puede inferir que la MT visita cada celda a lo sumo una vez, ya que no puede volver a celdas anteriores por no poder moverse a la izquierda.

Se puede construir una MT T que decide si la MT M para a partir de una cadena w de la siguiente forma:

1. Se simula la ejecución de M a partir de la cadena w.
2. Mientras el cabezal lea símbolos de la cadena w, es decir, los primeros n pasos, se simulan las transiciones de M normalmente. Si en algún momento $\delta(q, s)$ no está definida, entonces M se detiene y por lo tanto T acepta.
3. Cuando se termina la cadena de entrada w, el cabezal de M se encuentra en el primer símbolo blanco a la derecha de w. A partir de este punto, el comportamiento está totalmente determinado por el estado actual y el símbolo blanco, ya que el cabezal solo leerá símbolos blancos a partir de ahora. Como el conjunto de estados Q de M es finito, al cabo de a lo sumo $|Q| + 1$ pasos pasara una de dos cosas:
   i. Se encuentra una transición $\delta(q, B)$ que no está definida, lo cual implica que M se detiene, y por lo tanto T acepta.
   ii. Se repite un estado sin que M se haya detenido, lo cual implica que M entró en un loop, y por lo tanto T rechaza.

Como se ha construido una MT T que **acepta** si M para a partir de una cadena w, y **rechaza** en caso contrario y además nunca loopea, se concluye que el problema es decidible.

c. Probar que se puede decidir si dada una MT M, existe una cadena w a partir de la cual M para en a lo sumo 10 pasos. Ayuda: ¿Hasta qué tamaño de cadenas hay que chequear?

Se tiene una MT M y se quiere decidir si existe una cadena w a partir de la cual M para en 10 pasos como máximo.

Dado que M tiene que detenerse en 10 pasos o menos, el cabezal solo puede visitar a lo sumo 11 celdas de la cinta, la posición inicial + 10 movimientos. Las celdas a la izquierda de la cadena de entrada w siempre son blanco y por lo tanto son irrelevantes. Las celdas desde la posición inicial de w hasta la posición 10 pueden contener símbolos de w o B si $|w| < 10$. Por lo tanto el comportamiento de M en los primeros 10 pasos depende exclusivamente de los primeros 11 símbolos de w (si $|w| < 10$), entonces se asume que el resto de los símbolos de w son blancos).

Para decidir si existe algun w tq M para en a lo sumo 10 pasos, se deben chequear todas las cadenas w de longitud a lo sumo 10, y estas pueden ser generadas en el orden canónico típico.

Para cada una de estas cadenas se simula M a partir de esa cadena durante 10 pasos o hasta que pare, lo que pase primero. Si en alguna simulación M se detiene en el paso 10 o antes, entonces se acepta. En caso contrario se rechaza.

Como la cantidad de cadenas a chequear es finita y además cada simulación se lleva a cabo en tiempo finito, se concluye que el problema es decidible.

d. ¿Se puede decidir si dada una MT M, existe una cadena w de a lo sumo 10 símbolos a partir de la cual M para? Justificar la respuesta.

Esto no es decidible:

1. Se enumeran todas las cadenas w con $|w| \leq 10$ en el orden canónico típico. Es una cantidad **finita** de cadenas.
2. Se simula M a partir de cada una de estas cadenas en paralelo, es decir, se simula un paso de M a partir de cada cadena, luego se simula el segundo paso de M a partir de cada cadena, y así sucesivamente.
3. Si alguna de las simulaciones se detiene, entonces se acepta.

4. Pero si ninguna simulación hace que M se detenga entonces todas las simulaciones loopean, y es imposible detectar que esto es lo que está pasando en tiempo finito debido a la indecidibilidad del Halting Problem.

3. Sea M_1 una MT que genera un lenguaje recursivo L sin repetir cadenas y siguiendo el orden canónico. Probar que existe una MT M_2 que decide L. Ayuda: cuidado con el caso en que L sea finito.

M_2 es la MT que decide L y puede funcionar así:

1. Recibe una cadena w.
2. Simula M_1 paso a paso. Cada vez que M_1 genera una cadena v, se chequea si $v = w$.
   i. Si $v = w$, entonces M_2 acepta.
   ii. Si v está DESPUÉS que w en el orden canónico (esto se puede chequear fácilmente de forma mecánica) entonces M_2 rechaza.
3. Si M_1 se detiene (es decir que L es finito) y nunca se llegó a que $v = w$, entonces M_2 rechaza, porque ya se generaron todas las cadenas de L y w no era una de ellas.

- Si $w \in L$, entonces M_1 lo va a generar tarde o temprano, y por lo tanto M_2 lo va a aceptar.
- Si $w \notin L$, tenemos dos casos:
  - Si L es finito, entonces M_1 se detiene luego de generar todas las cadenas de L, y como w no era una de ellas, entonces M_2 rechaza.
  - Si L es infinito, como solo hay una cantidad finita de cadenas menores que w en el orden canónico y como L tiene infinitas cadenas, M_1 debe generar eventualmente una cadena mayor que w (no puede generar infinitas cadenas todas menores). Cuando esto sucede, M_2 rechaza.

4. Considerando la reducción de HP a L_U descripta en clase, responder:

a. Explicar por qué la función identidad, es decir la función que a toda cadena le asigna la misma cadena, no es una

reducción de HP a L_U.

La función identidad $f(x) = x$ no es una reducción de HP a L_U porque no cumple con la propiedad fundamental de las reducciones que es que para toda cadena w, si $w \in HP$ entonces $f(w) \in L_U$, y si $w \notin HP$ entonces $f(w) \notin L_U$. En particular, con la función identidad pasa lo siguiente:

1. Si $(\langle M \rangle, w) \notin HP$, entonces la máquina no se detiene a partir de w, y luego de aplicarle la función identidad se obtiene exactamente el mismo par tal que $(\langle M \rangle, w) \notin L_U$ porque M no acepta w (ya que no se detiene, y no detenerse implica no aceptar). Por lo tanto hasta ahora todo bien.
2. Si $(\langle M \rangle, w) \in HP$, entonces la máquina se detiene a partir de w, pero esto no necesariamente implica que acepta esa cadena, ya que se puede haber detenido en el estado q_R, lo cual viola la definición de L_U. Por ende $f(\langle M \rangle, w)$ no siempre pertenece a L_U y por lo tanto se rompe la reducción.

b. Explicar por qué las MT M_2 generadas en los pares de salida $(\langle M_2 \rangle, w)$, o bien paran aceptando, o bien loopean.

Lo que hace la función $f(\langle M_1 \rangle, w) = (\langle M_2 \rangle, w)$ en la reducción descripta en clase es cambiar todos los estados q_R de la máquina M_1 por estados q_A. Por lo tanto transforma la máquina original en una máquina que si antes rechazaba, ahora acepta. Sin embargo, esto no afecta para nada al caso en que la máquina loopeaba con la cadena w, ya que el cambio de estados no afecta a los loopeos. Por ende las máquinas M_2 generadas en los pares de salida $(\langle M_2 \rangle, w)$, o bien paran aceptando (si M_1 se detenía ya sea aceptando o rechazando), o bien loopean (si M_1 loopeaba).

El motivo de esta transformación es para garantizar que la propiedad fundamental de la reducción se cumpla:

1. Si $(\langle M_1 \rangle, w) \in HP$, entonces M_1 se detiene a partir de w, lo cual implica que M_2 se detiene en q_A a partir de w, y por lo tanto $f(\langle M_1 \rangle, w) = (\langle M_2 \rangle, w) \in L_U$.
2. Si $(\langle M_1 \rangle, w) \notin HP$, entonces M_1 no se detiene a partir de w, lo cual implica que M_2 tampoco se detiene a partir de w, y por lo tanto $f(\langle M_1 \rangle, w) = (\langle M_2 \rangle, w) \notin L_U$.

c. Explicar por qué la función utilizada para reducir HP a L_U también sirve para reducir HP^C a L_U^C.

$HP^C = \{(\langle M \rangle, w) \mid M \text{ no se detiene a partir de } w\}$

$L_U^C = \{(\langle M \rangle, w) \mid M \text{ no acepta } w\}$

Probando que la función utilizada para reducir HP a L_U también sirve para reducir HP^C a L_U^C:

1. Si $(\langle M_1 \rangle, w) \in HP^C$, entonces M_1 no se detiene a partir de w, lo cual implica que M_2 tampoco se detiene a partir de w, y por lo tanto $f(\langle M_1 \rangle, w) = (\langle M_2 \rangle, w) \in L_U^C$.
2. Si $(\langle M_1 \rangle, w) \notin HP^C$, entonces M_1 se detiene a partir de w, lo cual implica que M_2 se detiene en q_A a partir de w, y por lo tanto $f(\langle M_1 \rangle, w) = (\langle M_2 \rangle, w) \notin L_U^C$.

La razón fundamental de por qué la función también sirve acá es porque quedarse loopeando implica no aceptar, y por lo tanto esos casos están dentro de L_U^C.

d. Explicar por qué la función utilizada para reducir HP a L_U no sirve para reducir L_U a HP.

Analizando los dos casos:

1. Si $(\langle M_1 \rangle, w) \in L_U$, entonces M_1 acepta w, lo cual implica que M_2 se detiene en q_A a partir de w, y por lo tanto $f(\langle M_1 \rangle, w) = (\langle M_2 \rangle, w) \in HP$.
2. Si $(\langle M_1 \rangle, w) \notin L_U$, entonces M_1 no acepta w, pero esto no necesariamente implica que M_1 loopea a partir de w, ya que se puede haber detenido en el estado q_R. En este caso, M_2 se detiene en q_A a partir de w, y por lo tanto $f(\langle M_1 \rangle, w) = (\langle M_2 \rangle, w) \in HP$, lo cual rompe la reducción.

e. Explicar por qué la siguiente MT M_f no computa una reducción de HP a L_U: dada una cadena válida $(\langle M \rangle, w)$, M_f ejecuta M sobre w, si M acepta entonces genera la salida $(\langle M \rangle, w)$, y si M rechaza entonces genera la cadena 1.

Nuevamente analizando los dos casos:

1. Si $(\langle M \rangle, w) \in HP$, entonces M se detiene a partir de w. M_f ejecuta M sobre w, y si M acepta entonces genera la salida $(\langle M \rangle, w)$, pero si rechaza entonces genera la cadena 1. Como HP posee máquinas que se detienen tanto aceptando como rechazando, entonces M_f puede fácilmente dar como resultado la cadena 1 para una máquina que se detiene rechazando, y como $1 \notin L_U$ se rompe la reducción.
2. No es necesario analizar el caso en que $(\langle M \rangle, w) \notin HP$ porque ya se mostró que la función no es una reducción válida.

5. Sabiendo que $L_U \in RE$ y $L_U^C \in \text{CO-RE}$:

a. Probamos en clase que existe una reducción de L_U a L_{Σ^*}. En base a esto, ¿qué se puede afirmar con respecto a la ubicación de L_{Σ^*} en la jerarquía de la computabilidad?

1. Se sabe que $L_U \in (RE - R)$.
2. Se sabe por propiedad que si dados dos lenguajes L_1 y L_2 tales que $L_1 \leq L_2$ y $L_1 \notin R$, entonces $L_2 \notin R$.
3. Como se sabe que $L_U \notin R$ y se ha probado que existe una reducción de L_U a L_{Σ^*}, entonces se puede afirmar que $L_{\Sigma^*} \notin R$.
4. No se puede afirmar nada más, como por ejemplo si L_{Σ^*} está o no en RE.

b. Se prueba que existe una reducción de L_U^C a $L_\emptyset$. En base a esto, ¿qué se puede afirmar con respecto a la ubicación de $L_\emptyset$ en la jerarquía de la computabilidad?

1. Se sabe que $L_U^C \in (\text{CO-RE} - R)$ y además que $L_U^C \notin RE$.
2. Se sabe por propiedad que si dados dos lenguajes L_1 y L_2 tales que $L_1 \leq L_2$ y $L_1 \notin RE$, entonces $L_2 \notin RE$.
3. Como se sabe que $L_U^C \notin RE$ y se ha probado que existe una reducción de L_U^C a $L_\emptyset$, entonces se puede afirmar que $L_\emptyset \notin RE$.

6. Sea el lenguaje $D_{HP} = \{w_i \mid M_i \text{ para a partir de } w_i\}$ (considerar el orden

canónico}. Encontrar una reducción de D_{HP} a HP. Comentario: hay que definir la función de reducción y probar su total computabilidad y correctitud.

$D_{HP} = \{w_i \mid M_i \text{ para a partir de } w_i\}$
$HP = \{(\langle M \rangle, w) \mid M \text{ se detiene a partir de } w\}$

Se quiere encontrar una reducción $D_{HP} \leq HP$.

1. Definición de reducción:
   i. Si $w_i \in D_{HP}$, entonces $f(w_i) \in HP$.
   ii. Si $w_i \notin D_{HP}$, entonces $f(w_i) \notin HP$.
2. Se define una MT M_f que computa la función de reducción f, simbólicamente $f(w_i) = (\langle M_i \rangle, w_i)$.
3. La función f hace lo siguiente:
   i. Recibe una cadena w_i.
   ii. Genera cadenas $v_1, v_2, \ldots, v_j$ en el orden canónico hasta que $v_j = w_i$. De esta forma hallamos el índice i de la cadena original, que no lo sabíamos de antemano.
   iii. Genera MTs $M_1, M_2, \ldots, M_i$ en el orden canónico hasta llegar a M_i.
   iv. Devuelve la cadena $(\langle M_i \rangle, w_i)$.
4. Total computabilidad: la función f es total computable porque para toda cadena w_i se puede hallar su índice i generando cadenas en el orden canónico, y luego se puede generar la MT M_i también en el orden canónico. Por lo tanto, para toda cadena de entrada w_i, la función f devuelve una salida válida en tiempo finito, ya que sea cual sea w_i, se sabe que está en el orden canónico (porque éste contiene TODAS las cadenas), y por lo tanto se llegará a ella en tiempo finito.
5. Correctitud:
   i. Si $w_i \in D_{HP}$, entonces M_i se detiene a partir de w_i, lo cual implica que $(\langle M_i \rangle, w_i) \in HP$, y por lo tanto $f(w_i) \in HP$.
   ii. Si $w_i \notin D_{HP}$, entonces M_i no se detiene a partir de w_i, lo cual implica que $(\langle M_i \rangle, w_i) \notin HP$, y por lo tanto $f(w_i) \notin HP$.

Por lo tanto, $D_{HP} \leq HP$.

7. Sean $TAUT$ y $NOSAT$ los lenguajes de las fórmulas booleanas sin cuantificadores tautológicas

(satisfactibles por todas las asignaciones de valores de verdad) e insatisfactibles (ninguna asignación de valores de verdad las satisface), respectivamente.

Encontrar una reducción de $TAUT$ a $NOSAT$.

Comentario: hay que definir la función de reducción y probar su total computabilidad y correctitud.

$TAUT = \{\phi \mid \phi \text{ es una fórmula booleana sin cuantificadores que es tautología}\}$
$NOSAT = \{\phi \mid \phi \text{ es una fórmula booleana sin cuantificadores que es insatisfactible}\}$

Se quiere encontrar una reducción $TAUT \leq NOSAT$.

1. Definición de reducción:
   i. Si $\phi \in TAUT$, entonces $f(\phi) \in NOSAT$.
   ii. Si $\phi \notin TAUT$, entonces $f(\phi) \notin NOSAT$.
2. Se define una MT M_f que computa la función de reducción f, simbólicamente $f(\phi) = \neg\phi$.
3. La función f hace lo siguiente:
   i. Recibe una fórmula booleana sin cuantificadores ϕ.
   ii. Devuelve la fórmula $\neg\phi$, es decir, la negación de la expresión entera de ϕ, sin importar que tan compleja sea. Esto es porque se sabe que la negación de una tautología siempre es una contradicción, y toda contradicción es insatisfactible.
4. Total computabilidad: la función f es total computable porque para toda fórmula booleana sin cuantificadores ϕ se puede generar su negación $\neg\phi$ en tiempo finito, ya que se puede recorrer la expresión de ϕ y generar la expresión de $\neg\phi$ aplicando las reglas de la lógica proposicional.
5. Correctitud: la función f es correcta porque:
   i. Si $\phi \in TAUT$, entonces ϕ es una tautología, lo cual implica que $\neg\phi$ es una contradicción, y por lo tanto $\neg\phi \in NOSAT$, y $f(\phi) \in NOSAT$.
   ii. Si $\phi \notin TAUT$, entonces ϕ no es una tautología, lo cual implica que $\neg\phi$ no es una contradicción, y por lo tanto $\neg\phi \notin NOSAT$, y $f(\phi) \notin NOSAT$.

Por lo tanto, $TAUT \leq NOSAT$.

8. Se prueba que existe una reducción de L_U^C a L_{Σ^*} (y así, como $L_U^C \notin RE$, entonces se cumple que $L_{\Sigma^*} \notin RE$). La reducción es la siguiente. Para toda w : $f((\langle M_1 \rangle, w)) = \langle M_2 \rangle$, tal que M_2, a partir de su entrada v, ejecuta $|v|$ pasos de M_1 a partir de w, y acepta sii M_1 no acepta. Probar que la función definida es efectivamente una reducción de L_U^C a L_{Σ^*} (es total computable y correcta).

$L_U^C = \{(\langle M \rangle, w) \mid M \text{ no acepta } w\}$
$L_{\Sigma^*} = \{\langle M \rangle \mid L(M) = \Sigma^*\}$

La función de reducción que se da es $f((\langle M_1 \rangle, w)) = \langle M_2 \rangle$.

1. **Total computabilidad**: La función f dada es total computable porque dado $(\langle M_1 \rangle, w)$ se puede construir efectivamente una mt M_2 que, al recibir una cadena v, simula M_1 a partir de w durante $|v|$ pasos, y acepta si y solo si M_1 no acepta w en esos pasos. Esta construcción es mecánica y se puede realizar en tiempo finito.
2. **Correctitud**:
   i. Si $(\langle M_1 \rangle, w) \in L_U^C$, entonces M_1 nunca acepta w. Para cualquier cadena v, la simulación de M_1 durante $|v|$ pasos no encuentra ninguna aceptación, por lo que M_2 acepta v. Entonces tenemos $L(M_2) = \Sigma^*$, y $\langle M_2 \rangle \in L_{\Sigma^*}$.
   ii. Si $(\langle M_1 \rangle, w) \notin L_U^C$, entonces M_1 acepta w en algún paso t. Para v con $|v| \geq t$, la simulación detecta la aceptación y M_2 rechaza. Para v con $|v| < t$, M_2 acepta. Así $L(M_2) \neq \Sigma^*$, y por lo tanto $\langle M_2 \rangle \notin L_{\Sigma^*}$.